C3103

Computer Networks



Dr. Pradeepthi K V

C R Rao Advanced Institute of Mathematics, Statistics and Computer Science

University of Hyderabad Campus, Gachibowli,

Hyderabad.

UNIT - III

Network Layer

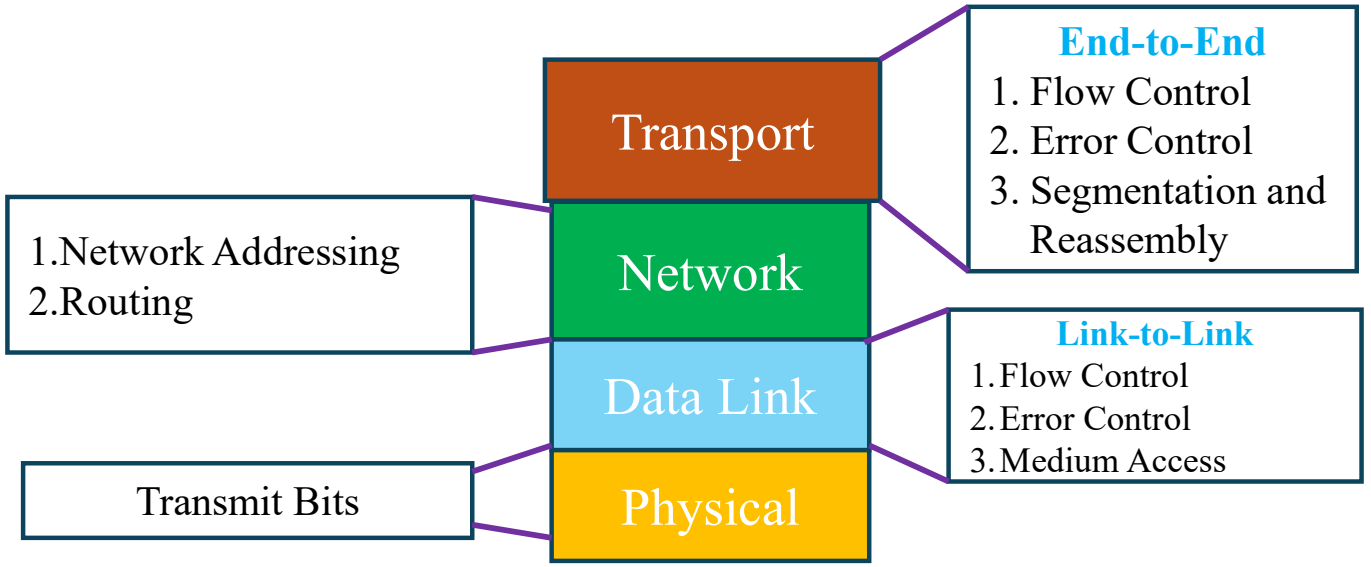
- Design issues,
- Routing algorithms: shortest path routing, Flooding, Hierarchical routing, Broadcast, Multicast, distance vector routing,
- Congestion Control Algorithms, Quality of Service,
- Internetworking, The Network layer in the internet.

Reference:

1. Computer Networks, 5th Edition by Andrew Tanenbaum & David Wetherall, Copyright. Pearson Education 2011.
2. Jim Kurose, Keith Ross, Computer Networking: A Top-Down Approach, 8th Edition, Pearson Education.

OSI Layers & Functionalities

Refresh



OSI L3 – Network Layer

Refresh

- Responsible for addressing messages across networks
- Translates network logical addresses into physical machine addresses
- Breaks large segments into smaller packets
- Determines routing across networks
- Supports communications between logically separate networks
    - Routers and gateways (devices that pass data from one network to another) operate in the Network layer

Congestion Control Algorithms

- Congestion ---
  - **Too many packets are present in the subnet, leading to delays and packet loss and degrades performance.**
- Congestion control algorithms aim to keep network load below the point of congestion.
- Keep network load below congestion point

Rate at which Useful packets are delivered in the system

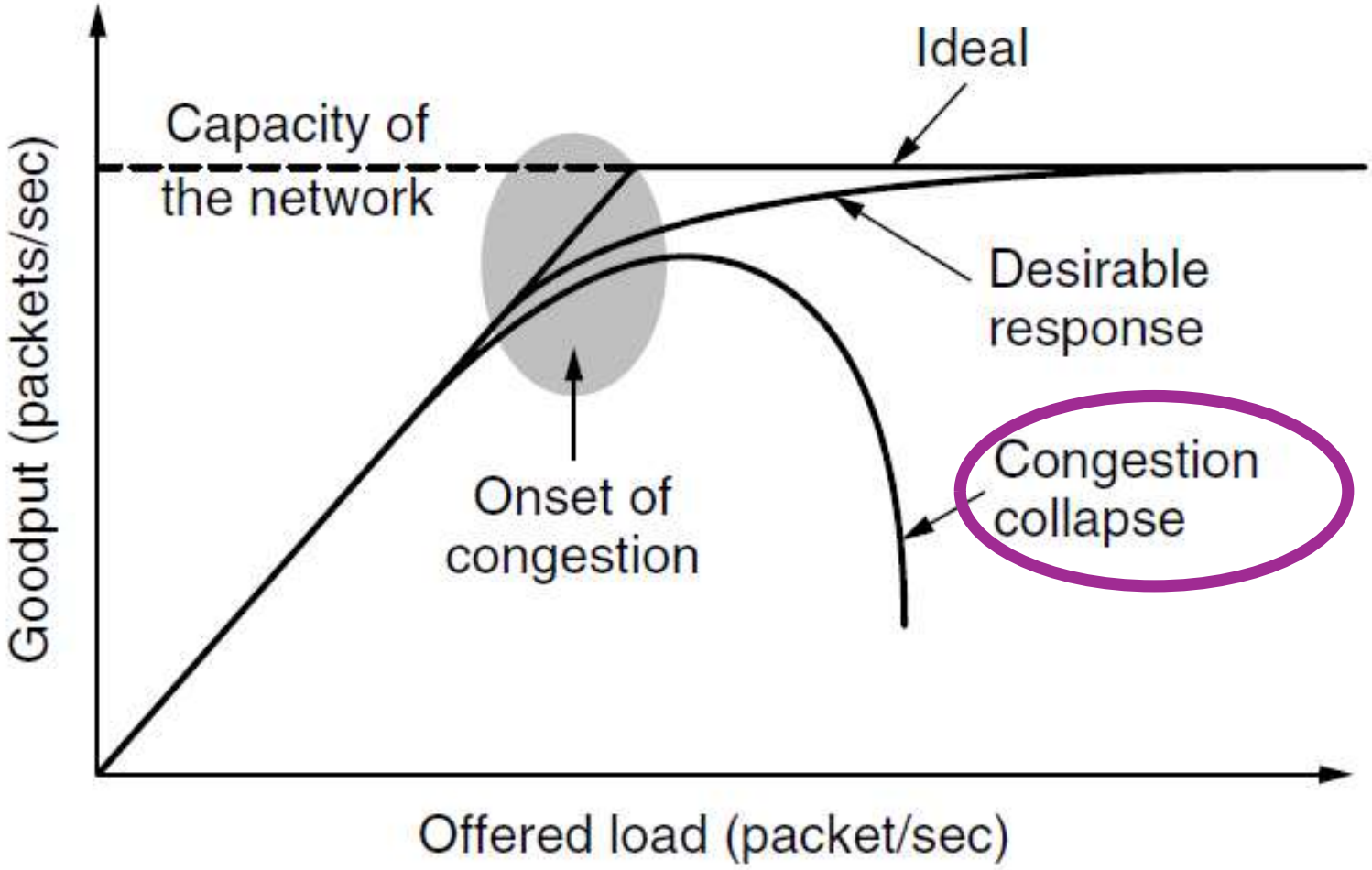


Figure 5-21. With too much traffic, performance drops sharply.

Avoiding Congestion – Key Points

Congestion and Congestion Collapse

- **Goal:**
  - Design networks to **avoid congestion where possible**.
  - Ensure they **do not collapse** if congestion occurs.
- **Reality:**
  - Congestion **cannot be completely avoided**.
  - Example: Multiple input streams all competing for the same output line → queues build up.
- **Memory Limitation:**
  - If memory is insufficient → packets are dropped.
  - If memory is infinite → problem gets **worse** (Nagle, 1987).
- **Why Infinite Memory Makes It Worse:**
  - Packets wait too long in queues.
  - By the time they reach the front, **they have already timed out**.
  - Senders retransmit, creating **duplicate packets**.
  - Leads to **congestion collapse** (network overwhelmed with duplicates).

- The simplest way to avoid congestion is to design a **network that matches the expected traffic load**.
- A **low-bandwidth link** on a heavily used path can easily cause congestion.

Flow Control

Approaches to Congestion Control

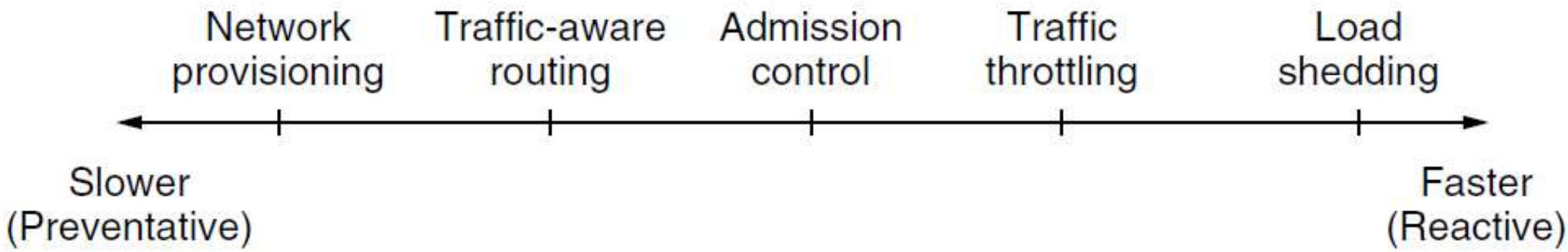


Figure 5-22. Timescales of approaches to congestion control.

Making the Most of Existing Network Capacity

In case of serious congestion, **resources can be added dynamically**, such as:

- Turning on spare routers
- Enabling backup lines normally reserved for fault tolerance
- Purchasing extra bandwidth on the open market

More commonly, **heavily utilized links and routers are upgraded** at the earliest opportunity.

This process is known as **provisioning**, which:

- Occurs on a time scale of months
- Is driven by **long-term traffic trends**

- Routes can be **adjusted to match traffic patterns** that vary during the day (due to different time zones).
- Example: **Changing shortest path weights** to shift traffic away from heavily used paths.
- Analogy: Like **radio helicopters reporting road congestion**, enabling drivers (packets) to avoid hotspots.
- This approach is called **traffic-aware routing**.
- **Splitting traffic across multiple paths** also helps balance load and reduce congestion.

Reducing Load to Control Congestion

- When **increasing capacity is not possible**, the only option is to **reduce the load**.
- In **virtual-circuit networks**, new connections can be **refused** if they may lead to congestion.
- This method is known as **Admission Control**.

Fine-Grained Congestion Control

- When congestion is imminent, the **network can provide feedback** to traffic sources.
  - Sources may be asked to **throttle traffic**, or the network may **slow it down itself**.
- **Challenges:**
  - **Detecting congestion onset**
    - Routers can monitor:
      - Average load
      - Queueing delay
      - Packet loss
    - Rising values indicate growing congestion.
  - **Notifying sources**
    - Routers must participate in a **feedback loop** with sources.
    - **Time scale tuning is critical:**
      - Too fast → oscillations (unstable system).
      - Too slow → sluggish response (ineffective).
    - Feedback must be **timely but stable**.
  - **Overhead concern**
    - Routers sending extra control messages can worsen congestion.
- **Last resort:**
  - The network may **discard packets it cannot deliver** → called **Load Shedding**.
  - A **good packet discard policy** helps prevent congestion collapse.

Congestion Control Methods

1. **Provisioning (Capacity Management)**
    1. Match network design to traffic load.
    2. Upgrade links/routers or add backup resources.
    3. Driven by long-term traffic trends.
2. **Traffic-Aware Routing**
    1. Adjust routes based on daily traffic patterns.
    2. Shift load away from congested paths (change path weights).
    3. Split traffic across multiple paths.
3. **Admission Control**
    1. Used in virtual-circuit networks.
    2. Refuse new connections if they would cause congestion.
    3. Prevents overload when capacity cannot be increased.
4. **Fine-Grained Feedback & Load Shedding**
    1. Routers detect congestion via load, delay, or packet loss.
    2. Provide feedback to sources to slow down traffic.
    3. Requires careful timing to avoid oscillations or delays.
    4. As a last resort → **discard packets (Load Shedding)** with good discard policy to prevent collapse.

Traffic-Aware Routing

- **Traditional routing schemes** use **fixed link weights**.
  - Adapt to topology changes.
  - Do **not adapt to load changes**.
- **Goal of traffic-aware routing**:
  - Shift traffic away from **hotspots** likely to experience congestion.
- **How it works**:
  - Link weight = function of:
    - Fixed link bandwidth
    - Propagation delay
    - Measured load / average queuing delay
  - Least-weight paths → favor lightly loaded links.
- **Example**:
  - Used in early Internet (Khanna & Zinky, 1989).
- **Problem (Oscillation Risk)**:
  - Traffic shifts from congested link (e.g., CF) to alternate link (e.g., EI).
  - Next update shows CF is free → traffic shifts back.
  - Results in **oscillating routing tables**, erratic paths, and instability.

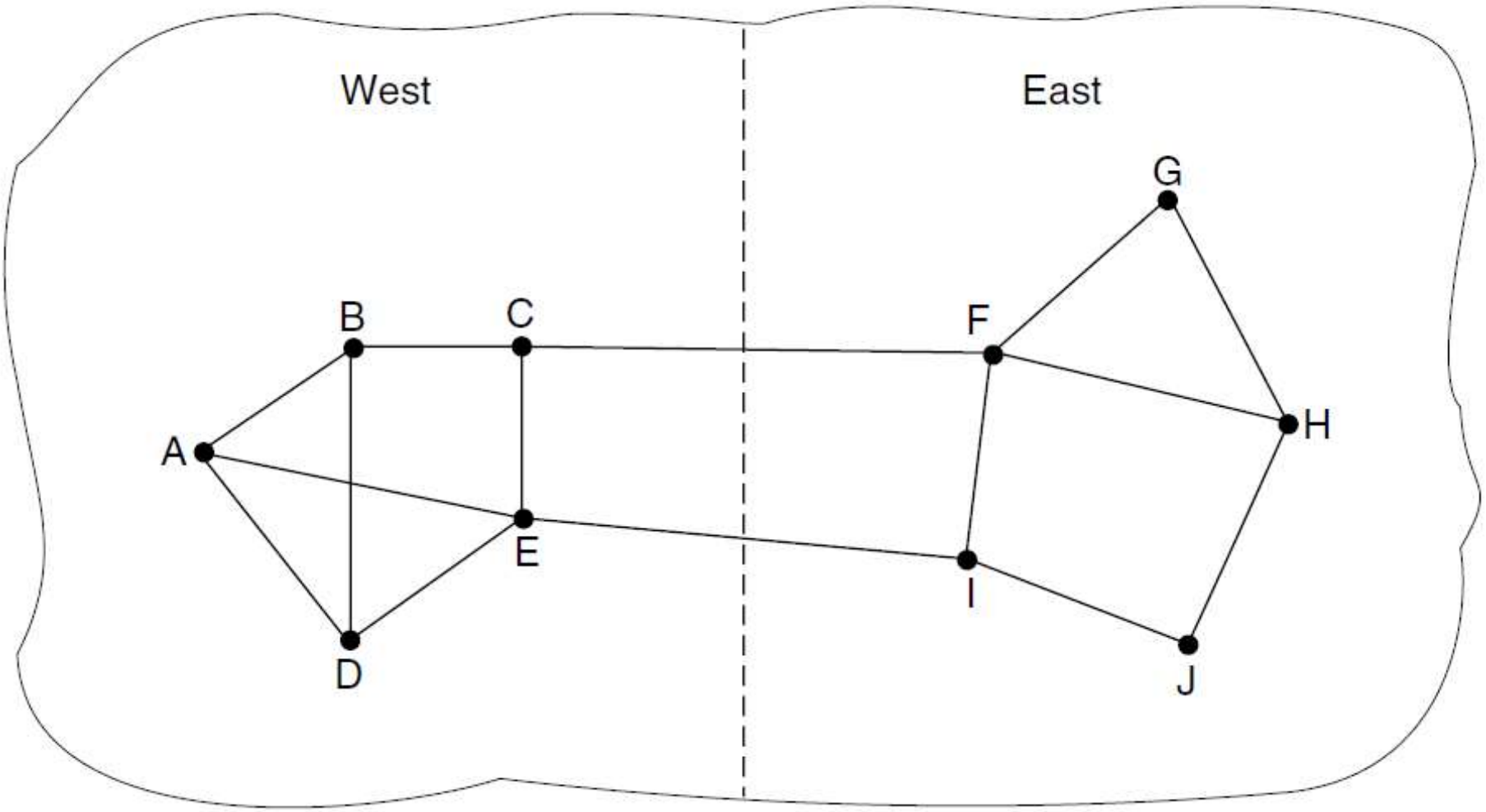


Figure 5-23. A network in which the East and West parts are connected by two links.

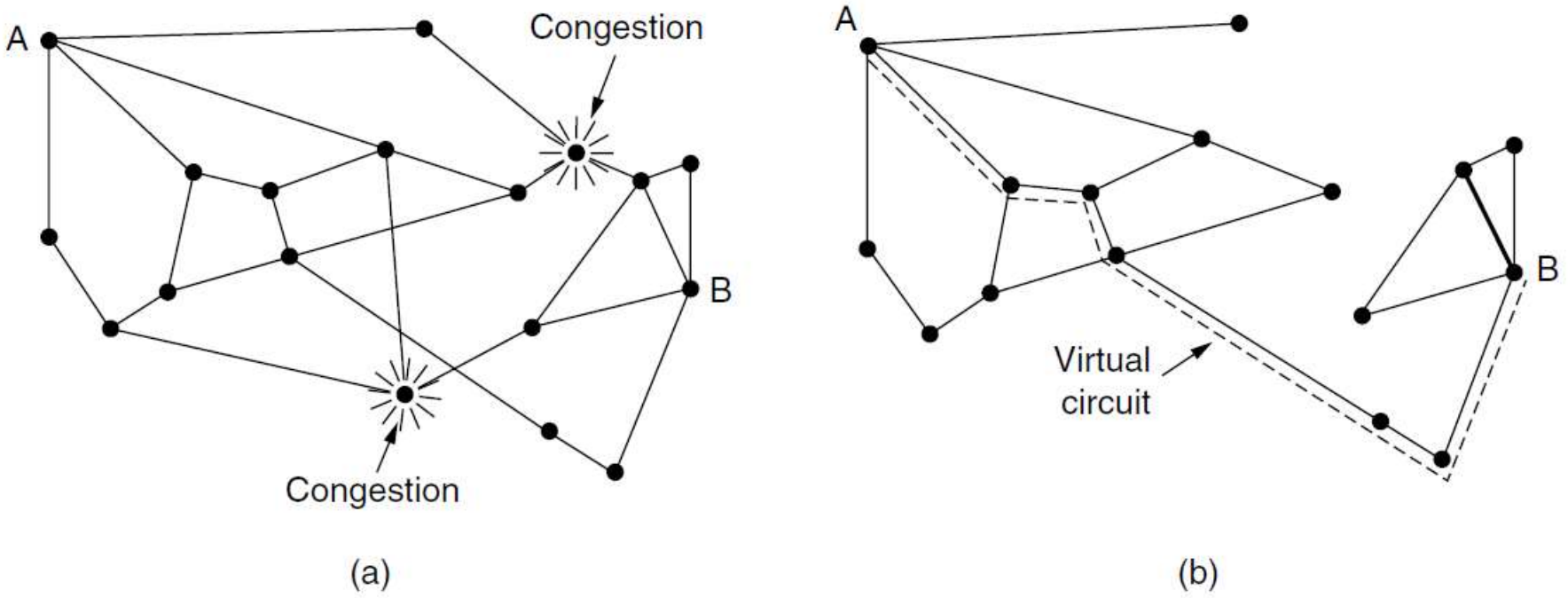
Avoiding Oscillations in Traffic-Aware Routing

- **Ignoring load** (considering only bandwidth + propagation delay):
  - Prevents oscillation problem.
  - But does not account for traffic variations.
- **Including load with narrow weight changes**:
  - Only slows down oscillations, does not eliminate them.
- **Two techniques for stable solutions**:
  - **Multipath routing**
    - Multiple paths from source → destination.
    - Traffic spread across both East–West links.
  - **Gradual traffic shifting**
    - Routing scheme shifts traffic **slowly** to allow convergence.
    - Example: Gallagher's scheme (1977).
- **Practical Internet approach**:
  - Routing protocols usually **do not adjust routes dynamically based on load**.
  - Instead, adjustments are made **outside the protocol** by slowly tuning its inputs.
  - This approach is called **Traffic Engineering**.

Admission Control in Virtual-Circuit Networks

- **Concept:**
  - Prevent congestion by **not setting up a new virtual circuit** unless the network can handle the added traffic.
  - If resources are insufficient → circuit setup **fails**.
  - Better than allowing overload, which worsens congestion.
- **Analogy:**
  - Telephone networks use admission control → when overloaded, switches **do not give dial tones**.
- **Challenge in Computer Networks:**
  - Telephone calls have fixed bandwidth (e.g., 64 kbps), making prediction easy.
  - Computer traffic is highly **variable** (different rates and sizes).
  - Circuits must include a **characterization of traffic** for admission control.
- **Traffic Description:**
  - Described by **rate** and **shape** of the flow.
  - Issue: traffic is often **bursty** (average rate is not enough).
    - Example: Web browsing (bursty) is harder to handle than video streaming with same average throughput.

- **Solution – Leaky Bucket / Token Bucket:**
  - Provides a simple traffic descriptor.
  - Two parameters:
    - **Average rate**
    - **Instantaneous burst size**
  - Widely used in **Quality of Service (QoS)** mechanisms.



Load Sensitivity Routing

Figure 5-24. (a) A congested network. (b) The portion of the network that is not congested. A virtual circuit from *A* to *B* is also shown.

Traffic Throttling

- **Approaches to Throttling Traffic (Datagram & Virtual-Circuit Networks)**
- **Goal:** Control traffic to prevent congestion.
- **Two main problems to solve:**
  - **Detecting approaching congestion** (before it fully arrives).
  - **Taking corrective action** to slow down traffic.
- **Router's Role:** Continuously monitor resource usage.
- **Key indicators of congestion:**
  1. **Utilization of output links** (how busy the outgoing lines are).
  2. **Buffer occupancy** (amount of queued packets inside the router).
  3. **Packet loss** (caused by insufficient buffer space).
- **Most useful indicator:**
  - **Buffer occupancy** is generally the best measure of imminent congestion.

Detecting Congestion in Routers

- **Utilization averages**
  - Do **not reflect burstiness** of traffic.
  - Example: 50% utilization is safe for smooth traffic but risky for highly variable traffic.
- **Packet loss counts**
  - Are a **late indicator** → congestion has already occurred once packets are dropped.
- **Queueing delay (preferred measure):**
  - Directly reflects congestion experienced by packets.
  - Normally should be **low**, but spikes during traffic bursts (backlog).
- **Estimation method:**
  - Take periodic **samples of instantaneous queue length (s)**.
  - Update the **average queueing delay (d)** using the sampled values.

$$d_{\text{new}} = \alpha d_{\text{old}} + (1 - \alpha)s$$

- where the constant α determines how fast the router forgets recent history.
- This is called an **EWMA** (**Exponentially Weighted Moving Average**).
- It smoothes out fluctuations and is equivalent to a low-pass filter.
- Whenever *d* moves above the threshold, the router notes the onset of congestion.

•Challenge:
- Congestion occurs **inside the network**, but it can only be relieved by **senders adjusting their traffic**.

•Two key requirements for routers:
- **Identify the responsible senders** causing congestion.
- **Deliver timely feedback** to them.

•Constraints:
- Feedback must be sent **carefully**, without injecting too many extra packets into the already congested network.

•Approach:
- Different congestion-control schemes use **different feedback mechanisms** (to be described next).

Choke Packets (Direct Feedback Mechanism)

- **Basic Idea:**
  - Router notifies the sender **directly** about congestion.
  - A special control message called a **choke packet** is sent back to the source.
- **How It Works:**
  - Router detects congestion.
  - It selects a congested packet and sends a **choke packet** to the **source host** (with destination info).
  - The original packet is **tagged** (header bit set) so that no further choke packets are generated for it.
  - Router may send choke packets at a **limited rate** to avoid worsening congestion.
- **Action at Source Host:**
  - On receiving a choke packet, source must **reduce traffic** to that destination (e.g., by 50%).
  - In datagram networks, **fast senders** are more likely to be targeted (since their packets dominate the queue).
  - If multiple choke packets arrive, the host **ignores extra ones** until the reduction interval passes.
  - If more choke packets arrive afterward → indicates congestion persists, further throttling may be needed.

- **Example in Internet History:**
  - Early Internet used the **SOURCEQUENCH message** (RFC 792, Postel 1981).
  - Did not succeed due to vague specification and inconsistent behavior.
  - Modern Internet uses **alternative notification methods** (e.g., ECN).

Explicit Congestion Notification (ECN)

- **Concept:**
  - Instead of sending extra packets (like choke packets), routers **mark existing packets** to indicate congestion.
  - Achieved by setting **special bits in the packet header**.
- **Working:**
  - Packets are sent with ECN bits **unset**.
  - If a router along the path is congested → it **marks the packet** as having experienced congestion.
  - When the packet reaches the **destination**, the mark is noticed.
  - The destination **echoes the congestion mark** back to the **sender** in its next reply (e.g., via TCP).
  - Sender then **reduces transmission rate**, similar to choke packet response.
- **Advantages:**
  - No extra control packets are generated (avoids worsening congestion).
  - Feedback is embedded within normal data flow.
  - Faster and more efficient than choke packets.
- **Implementation:**
  - Uses **2 bits in the IP header** to record congestion status.
  - First introduced in **binary feedback scheme** (Ramakrishnan & Jain, 1988, DECNET).
  - Standardized in the Internet (Ramakrishnan et al., 2001).

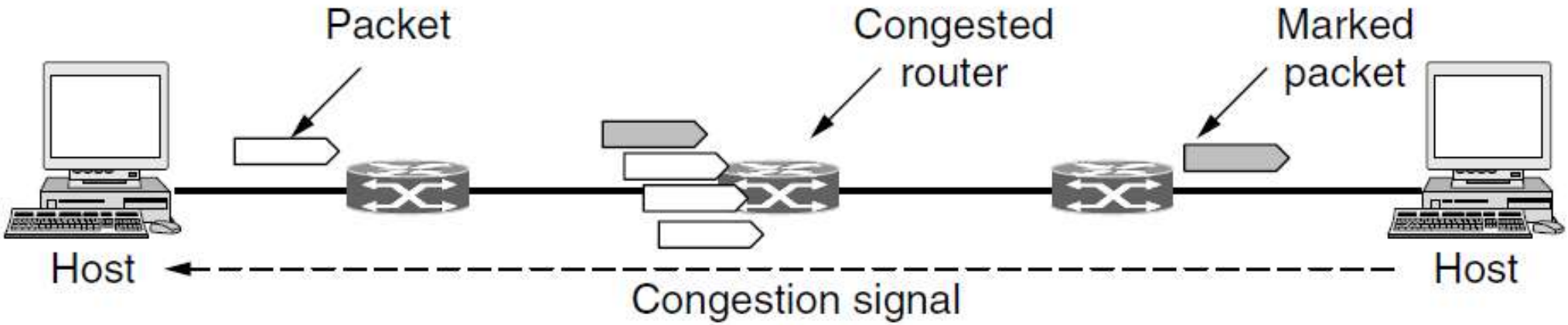
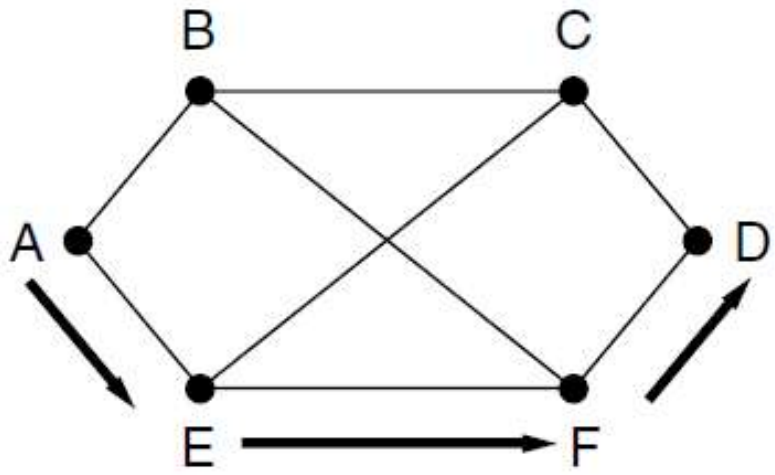
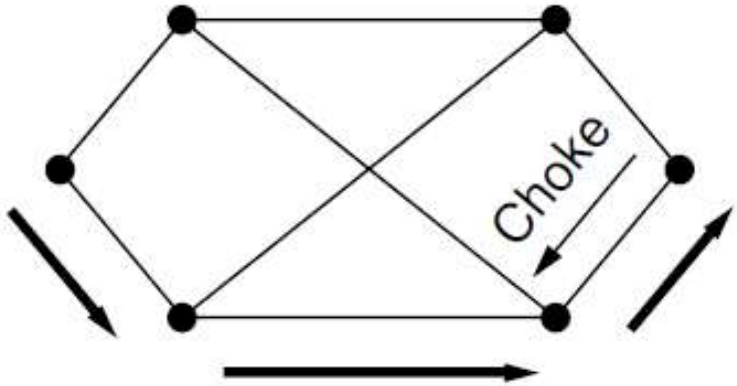


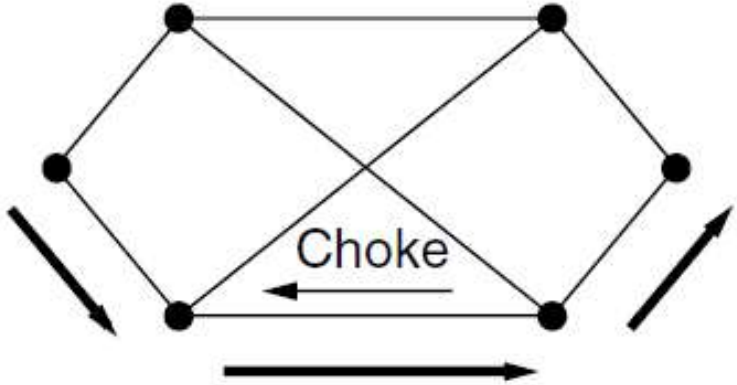
Figure 5-25. Explicit congestion notification

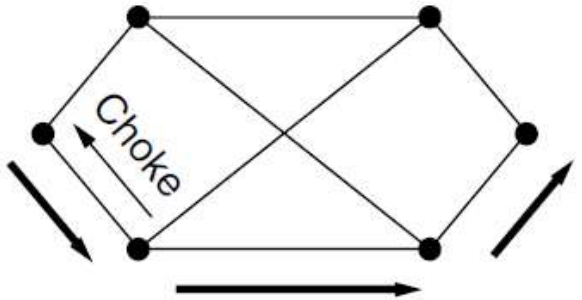
- **Delay Problem in Congestion Feedback**
- At **high speeds/long distances**, congestion feedback takes time to reach the sender.
- Example:
  - Host in **San Francisco (A)** sends data to **New York (D)** at **155 Mbps (OC-3)**.
  - If D runs out of buffers → choke packet takes **~40 ms** to reach A.
  - In that time, **6.2 Mb** of extra traffic is already in transit.
  - Even if A stops immediately, this backlog still reaches D.

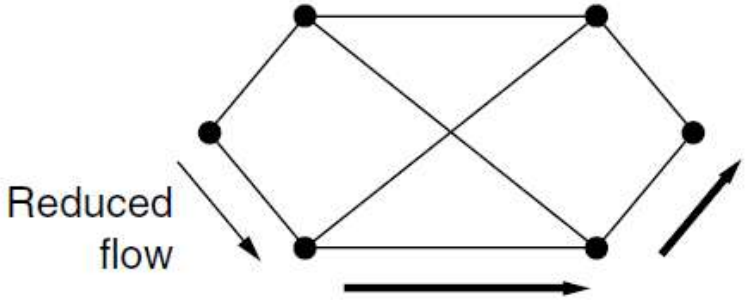
- **Hop-by-Hop Choke Packet Scheme**
- Instead of waiting until the choke packet reaches the **original sender**, each router along the path reacts:
  - Choke packet reaches **F** → F reduces flow to D.
    - F uses more buffers temporarily but relieves D immediately.
  - Choke packet reaches **E** → E reduces flow to F.
    - More buffers needed at E, but F is relieved.
  - Choke packet reaches **A** → Source finally slows down.

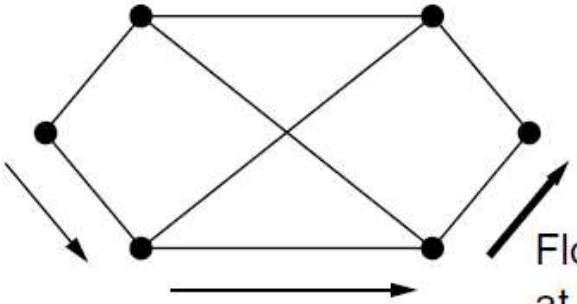

B
C
A
D
E
F

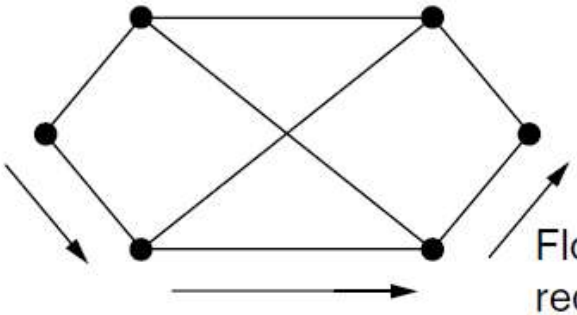

Choke


Choke

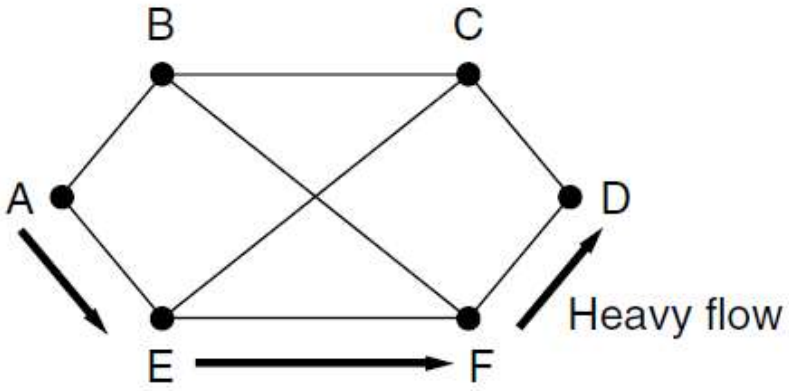

Choke

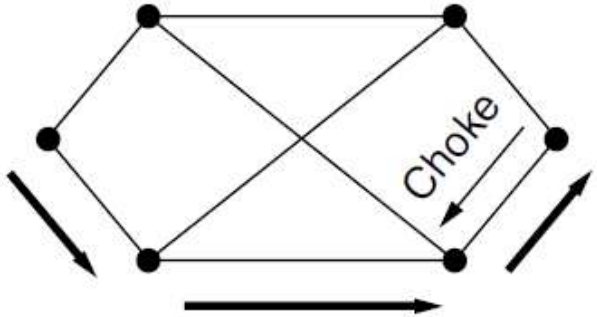

Reduced
flow

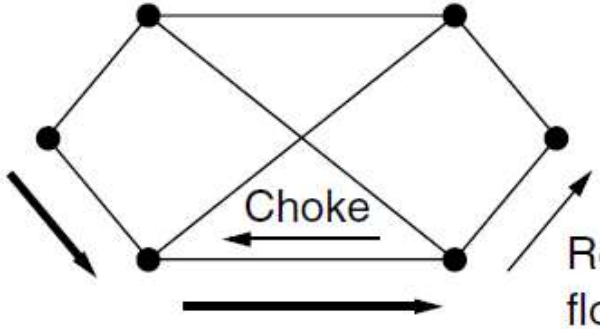

Flow is still
at maximum rate

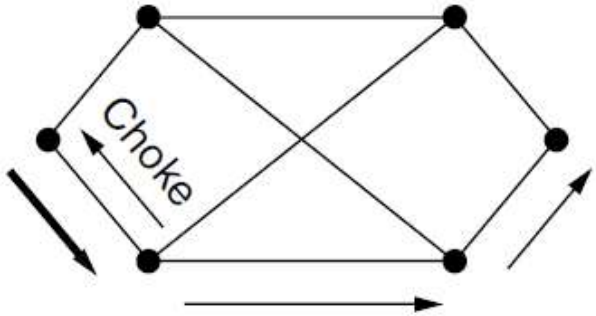

Flow is
reduced

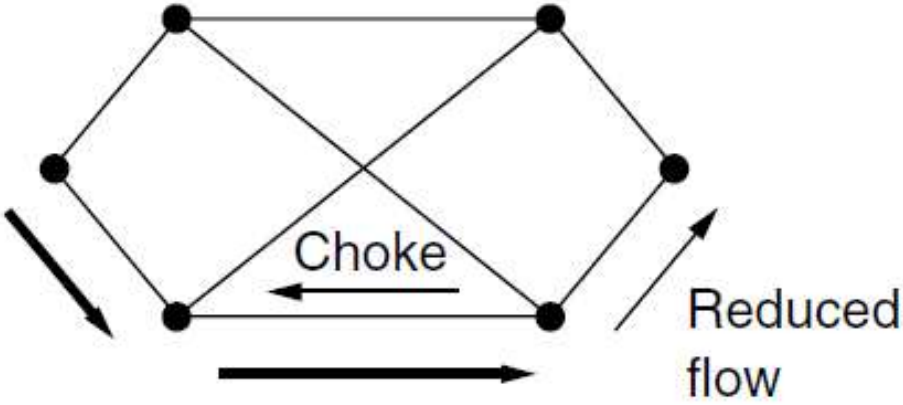
(a)

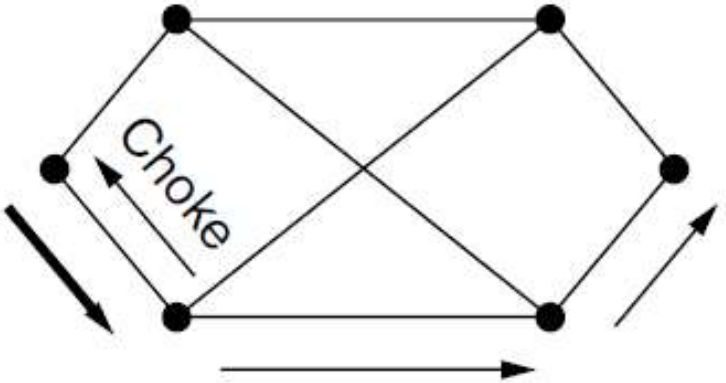

B
C
A
D
Heavy flow
E
F

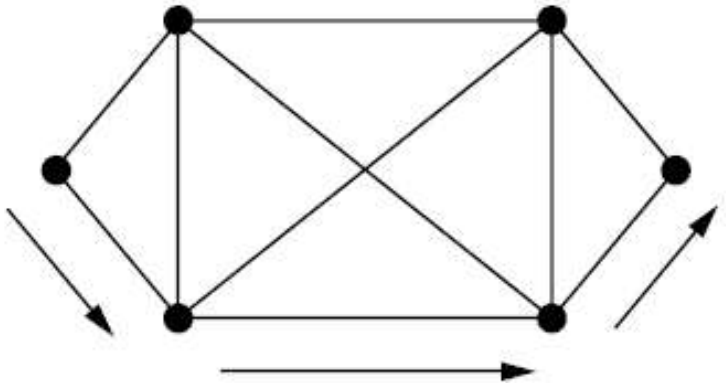

Choke


Choke
Reduced
flow


Choke


Choke
Reduced
flow


Choke



Advantages

- **Immediate relief** at congestion point.
- Prevents packet loss by managing flow locally.
- **Congestion is contained quickly** before it spreads.
- **Trade-off**
- **Higher buffer usage** at intermediate routers.
- But avoids dropping packets and stabilizes flow faster.

Load Shedding: Concept

- Final method for congestion control when other methods fail.
- **Definition**: Routers discard packets they cannot handle.
- **Origin**: Borrowed from electrical power systems (intentional blackouts to protect the grid).
-

- **Key Question: Which Packets to Drop?**
- Depends on **application type**:
  - **File transfer**:
    - *Old packets more valuable than new*.
    - Dropping an old packet forces unnecessary buffering.
    - Policy: **Wine** → *Old is better than new*.
  - **Real-time media**:
    - *New packets more valuable than old*.
    - Late packets are useless if they miss playback time.
    - Policy: **Milk** → *New is better than old*.

- **More Intelligent Load Shedding**
- Needs **sender cooperation**.
- Examples:
  - **Routing packets** → more important than regular data (loss may break connectivity).
  - **MPEG video** →
    - Full frame (I-frame) = very important.
    - Difference frames (P/B-frames) = less important.
    - Prefer dropping difference frames over full frames.
- **Packet Prioritization**
- Applications can **mark packets with importance levels**.
- Routers discard from the least important class first.

- **Avoiding Abuse**
- Without incentives, all packets may be marked “VERY IMPORTANT.”
- **Solution**: Pricing/accounting mechanisms.
  - Example: Allow faster sending if extra packets are marked *low priority*.
  - Efficient use of idle resources but no guaranteed right during congestion.

Random Early Detection (RED)

- **1. Key Idea**
- Better to deal with congestion **early** than wait until buffers are full.
- *Twist on load shedding*: Drop packets **before** buffer space is exhausted.
- **2. Motivation**
- Many Internet hosts **do not use ECN** → they only detect congestion via **packet loss**.
- TCP assumes:
  - Wired networks → reliable, so **loss = congestion** (not transmission errors).
  - Wireless errors are handled at **link layer**, so not visible to TCP.
- Thus, **packet loss = implicit congestion signal** to TCP → sender slows down.

- **3. How RED Works**
- Routers keep a **running average of queue length**.
- If the average exceeds a **threshold** → router is "congested."
- Instead of waiting for full buffers:
  - **Randomly drop a small fraction of packets**.
  - Random dropping → more likely to hit fast senders (they contribute most packets).
- Sender detects loss via missing ACK → **slows down transmission**.

4. Benefits of RED

•Prevents congestion collapse by acting early.

•Works like a **choke packet**, but implicitly (via loss).

•Improves performance compared to routers that drop packets **only when buffers overflow**.

5. Challenges / Tuning

•Needs careful tuning:

- *How many packets to drop?*
- Depends on number of senders that need to be notified.

6. ECN vs RED

•**ECN (Explicit Congestion Notification):** Preferred if available.

- Explicit marking → cleaner feedback.

•**RED:** Fallback when explicit signaling not possible.

Congestion Control in Networks –Summary

- **1. Traffic-Aware Routing**
- Adjusts routes based on load.
- Shifts traffic away from hotspots.
- Risk: oscillations if routes change too quickly.
- **2. Admission Control**
- Used in virtual-circuit networks.
- Rejects new connections if they would cause congestion.
- Needs traffic characterization (rate, burstiness – e.g., token bucket).

- **3. Congestion Avoidance**
- Operate near congestion point, not after collapse.
- Routers monitor utilization, buffer occupancy, queue delay.
- **4. Feedback to Senders**
- **Choke Packets**: Router sends control packet to source to slow down.
- **Hop-by-Hop Choking**: Intermediate routers also react, giving quick relief.
- **ECN (Explicit Congestion Notification)**: Packets marked instead of dropped; destination echoes mark to sender.

Congestion Control in Networks –Summary

5. Load Shedding

Routers drop packets when overloaded.

Policies:

Wine (old > new) → file transfers.

Milk (new > old) → real-time media.

Intelligent shedding: packet importance marking.

6. Random Early Detection (RED)

Drop packets **before buffers overflow**.

Probabilistic drops → faster senders more affected.

Implicit congestion signal (via TCP loss).

ECN preferred when available.

- **Congestion control =**

Prevent overload

+ Give timely feedback

+ Adjust sender rates

+ Drop smartly if needed.

QUALITY OF SERVICE

- **Quality of Service (QoS) in Networks**
- **Why QoS?**
- Basic congestion-control techniques → reduce congestion, improve performance.
- Some applications (e.g., **multimedia**) need **minimum throughput** & **maximum latency**.
- Internet is evolving to meet such requirements.
- **Naïve Solution: Overprovisioning**
- Build network with enough capacity for all possible traffic.
- Benefits:
  - Low loss, low latency.
  - Simple & reliable (like telephone dial tone).
- Drawbacks:
  - Very **expensive**.
  - Based on expected traffic → fails if traffic patterns change.

QoS Mechanisms (Smarter Approach)

- Provide **performance guarantees** at lower cost.
- Allow network to **honor guarantees** even under traffic spikes (by rejecting some requests).
- **Four Key QoS Issues**
- **Application Requirements** – Throughput, delay, jitter, reliability.
- **Traffic Regulation** – Control how traffic enters the network.
- **Resource Reservation** – Routers reserve capacity for flows.
- **Admission Control** – Decide if more traffic can be accepted without harming guarantees.

QoS Approaches in the Internet

•No **single technique** solves all QoS issues efficiently.

•**Multiple techniques** at the **network & transport layers** are combined.

•Two main Internet QoS models:

- **Integrated Services (IntServ)** – Per-flow guarantees with resource reservation.
- **Differentiated Services (DiffServ)** – Class-based service differentiation, scalable.

Flows and QoS Parameters

- **Flow**: Stream of packets from a source → destination (Clark, 1988).
  - In connection-oriented networks → all packets of a connection.
  - In connectionless networks → packets between two processes.
- **QoS Parameters (per flow):**
  - **Bandwidth** – Minimum throughput needed.
  - **Delay** – Maximum tolerable latency.
  - **Jitter** – Variation in delay.
  - **Loss** – Acceptable packet loss rate.
- **Key Insights:**
  - Apps can sometimes **tolerate/repair**:
    - **Loss** (via retransmission).
    - **Jitter** (via buffering).
  - Apps **cannot tolerate**:
    - **Too little bandwidth**.
    - **Excessive delay**.

Application	Bandwidth	Delay	Jitter	Loss
Email	Low	Low	Low	Medium
File sharing	High	Low	Low	Medium
Web access	Medium	Medium	Low	Medium
Remote login	Low	Medium	Medium	Medium
Audio on demand	Low	Low	High	Low
Video on demand	High	Low	High	Low
Telephony	Low	High	High	Low
Videoconferencing	High	High	High	Low

Figure 5-27. Stringency of applications' quality-of-service requirements.

- The variation (i.e., standard deviation) in the delay or packet arrival times is called **jitter**.

Categories of QoS (from ATM model, useful beyond ATM)

- **Constant Bit Rate (CBR)**
  - Example: Telephony.
  - Simulates a dedicated wire → uniform bandwidth & delay.
- **Real-Time Variable Bit Rate (rt-VBR)**
  - Example: Compressed videoconferencing.
  - Bandwidth varies (depends on video detail).
- **Non-Real-Time Variable Bit Rate (nrt-VBR)**
  - Example: Movies on demand.
  - Buffering at receiver absorbs jitter → not strictly real-time.
- **Available Bit Rate (ABR)**
  - Example: File transfer, email.
  - Delay/jitter insensitive, uses whatever bandwidth is available.

Traffic Shaping & Policing in QoS

- **Why needed?**
  - Data network traffic is **bursty** (not constant like voice calls).
  - Bursts can **overflow buffers → packet loss & delay**.
- **Traffic Shaping**
  - Technique to **regulate average rate & burstiness** of flows.
  - User & provider agree on a **traffic contract / SLA**.
  - Ensures predictable load on the network.
- **Service Level Agreement (SLA)**
  - Defines the allowed **traffic pattern (shape)**.
  - Provider guarantees delivery if contract is met.
- **Traffic Policing**
  - Provider checks if user respects SLA.
  - Excess packets → **dropped** or **marked low priority**.
- **Importance**
  - Crucial for **real-time flows** (voice, video).
  - Less critical for elastic traffic (e.g., file transfer, P2P).

Traffic Shaping Algorithms

- **Goal:** Regulate data transmission rate & burstiness to ensure QoS.

1. Leaky Bucket Algorithm

- Works like a bucket with a small hole.
- Packets enter the bucket → leak out at a constant rate.
- If bucket overflows → packets are discarded.
- **Effect:** Smooths traffic to a fixed, steady rate (no bursts).

2. Token Bucket Algorithm

- Tokens are generated at a constant rate and stored in a bucket.
- A packet can only be sent if tokens are available (1 token = 1 unit of data).
- Allows bursts as long as tokens exist; if empty → packets wait or are dropped.
- **Effect:** Controls average rate while allowing short bursts.
- **Comparison:**
- Leaky Bucket → strict rate control, eliminates bursts.
- Token Bucket → more flexible, allows bursts while maintaining long-term rate.

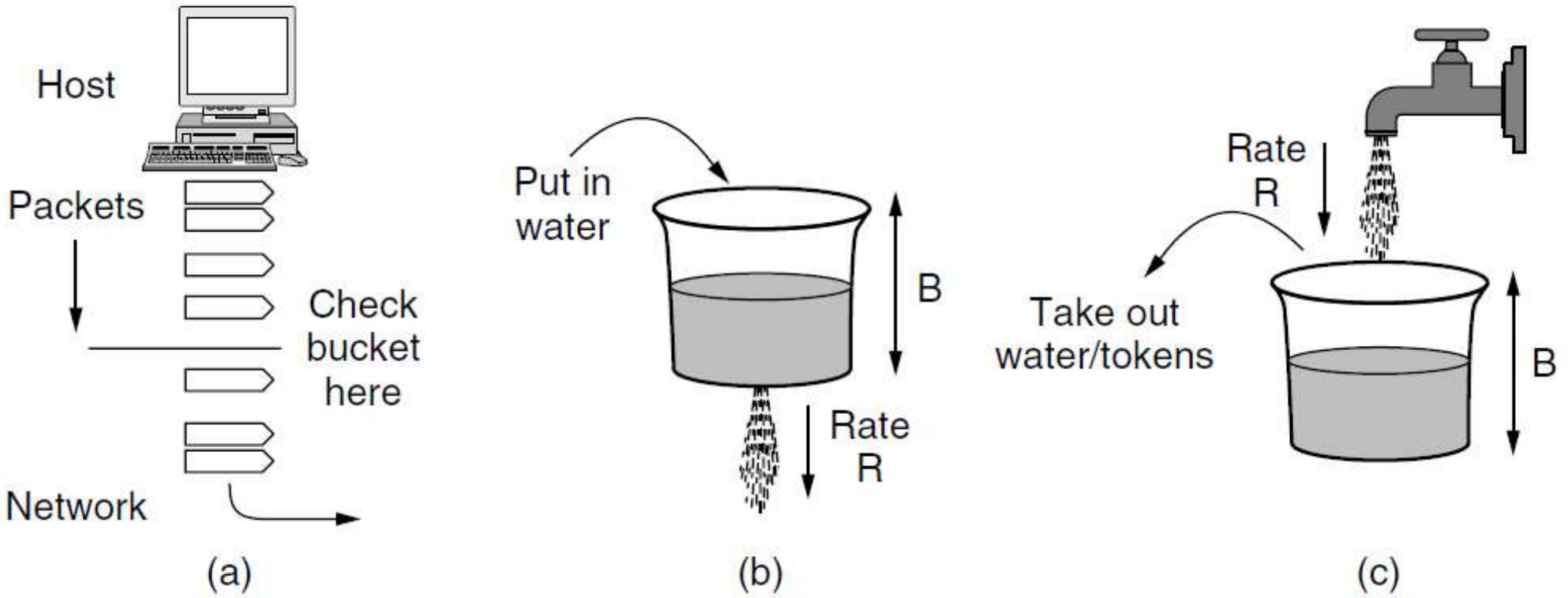


Figure 5-28. (a) Shaping packets. (b) A leaky bucket. (c) A token bucket.

Packet Scheduling

- To provide **performance guarantees**, networks must **reserve resources** along the packet route.
- Packets of a flow must follow the **same route** (like a virtual circuit) for guarantees to hold.
- **Packet scheduling algorithms** allocate router resources among flows.
- Three main resources to reserve:
  1. **Bandwidth** – prevent oversubscription (e.g., 1 Mbps flow needs dedicated bandwidth).
  2. **Buffer space** – absorb bursts; reserved buffers prevent packet loss for critical flows.
  3. **CPU cycles** – router processing resources needed for packet handling.

Packet Scheduling & Resource Management

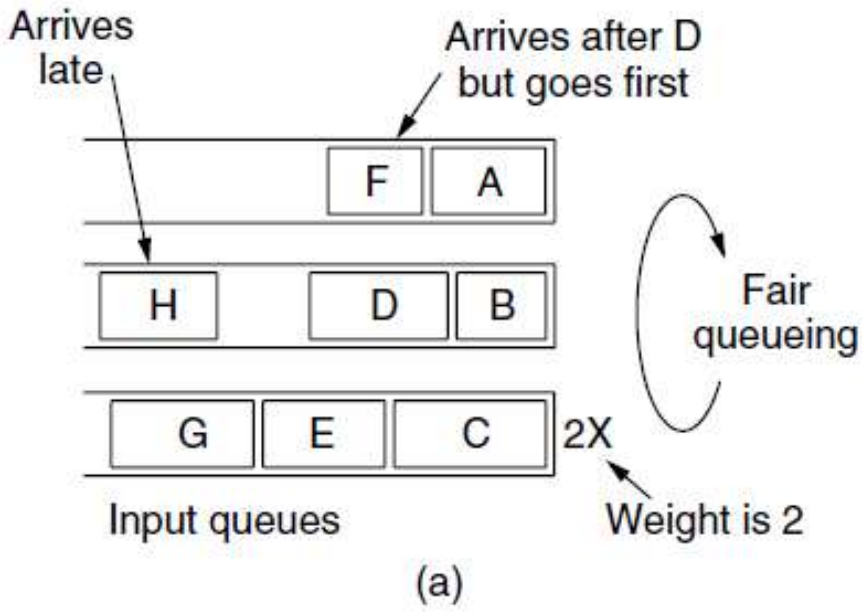
- **Resource Constraints** in routers:
  - **Bandwidth** – prevent oversubscription.
  - **Buffer space** – absorb bursts; reserved buffers ensure fairness.
  - **CPU cycles** – required for packet processing (e.g., ICMP packets need more CPU).
- **Packet Scheduling Algorithms** decide which buffered packet to send next.
- **FIFO / FCFS (First-In First-Out / First-Come First-Serve):**
  - Packets queued per output line, sent in arrival order.
  - **Tail drop**: new packets dropped when queue is full.
  - **Problem:** unfair—aggressive flows can hog capacity, causing delays and poor QoS for others.

- **RED (Random Early Detection):**
  - Drops packets randomly when average queue length grows.
  - Prevents queue overflow and spreads packet loss across flows.
- **Fair Queueing (FQ):**
  - **Separate queue per flow** for each output line.
  - Router serves flows in **round-robin** fashion.
  - Each host fairly gets 1/n of the output capacity when n flows compete.
  - **Flaw:** favors flows with large packets (since one “turn” can send more data).
- **Improved Fair Queueing (by Demers et al., 1990):**
- Adjusts round-robin to account for **packet size**, ensuring fair bandwidth distribution.

- **Byte-by-Byte Round-Robin (Fair Queueing Improvement)**
- **Problem with packet-by-packet round robin:**
  - Large packets gain more bandwidth than small packets.
- **Solution: Byte-by-byte simulation:**
  - Concept: Imagine **1 byte drained per round from each active flow's queue**.
  - Ensures fairness regardless of packet size.
- **Virtual Time Calculation:**
  - Each packet is assigned a **finish time** (virtual round number).
  - **Finish time = Start time + Packet length (L)**
  - **Start time =**
    - Finish time of the **previous packet** in that flow, OR
    - Arrival time if the flow's queue is empty.
- **Scheduling Rule:**
  - Packets across flows are sorted by **earliest finish time**.
  - Sent in that order → ensures **fair bandwidth allocation**.
- **Key Property:**
  - Approximates **bit-by-bit round-robin fairness** while still sending entire packets.



Figure 5-30. Round-robin fair queueing.



(a)

Packet	Arrival time	Length	Finish time	Output order
A	0	8	8	1
B	5	6	11	3
C	5	10	10	2
D	8	9	20	7
E	8	8	14	4
F	10	6	16	5
G	11	10	19	6
H	20	8	28	8

(b)

Figure 5-31. (a) Weighted Fair Queueing. (b) Finishing times for the packets.

Weighted Fair Queueing (WFQ)

- **Motivation:**
  - Some flows (e.g., video streaming) deserve more bandwidth than others (e.g., file transfers).
  - Solution: assign **weights (W)** to flows.
- **Weighted Service:**
  - A flow with weight W gets **W bytes per round**.
  - Larger weights = more bandwidth share.
- **Finish Time Formula (WFQ):**

$$F_i = \max(A_i, F_{i-1}) + L_i / W$$

Where:

- F_i =Finish time of the ith packet
- F_{i-1} =Finish time of the previous packet in the same flow
- A_i =Arrival time of the packet
- L_i =Packet length (bytes)
- W = Weight of the flow

- **Behavior:**
- Flows with higher weights get proportionally more bandwidth.
- Example: bottom queue in Fig. 5-31(a) with weight = 2 → packets finish faster.
- **Implementation Complexity:**
- Requires maintaining packets in **sorted order by finish time**.
- With NNN flows → **O(log N)** per packet (expensive at high speeds).
- **Optimization:**
- **Deficit Round Robin (DRR)** (Shreedhar & Varghese, 1995):
  - Approximation of WFQ.
  - Achieves fairness with only **O(1) operations per packet**.
  - Widely used in high-speed routers.

Priority Scheduling

- **Mechanism:**
  - Each packet has a priority level.
  - Higher-priority packets always served before lower-priority packets.
  - Within same priority: FIFO order.
- **Drawback:**
  - Risk of **starvation**: bursts of high-priority packets can indefinitely delay low-priority traffic.
- **Improvement via WFQ:**
  - Model high vs. low priority as a **two-queue WFQ system**.
  - Assign high priority a **large weight (e.g., 3)** → ensures most are served quickly.
  - Still guarantees some service for low-priority packets.
  - High priority ≈ **infinite weight case**.

- **Timestamp-Based Scheduling (Clark et al., 1992)**
- **Mechanism:**
  - Packets carry timestamps showing how far ahead/behind schedule they are.
  - Scheduling order = ascending timestamp.
- **Effect:**
  - Packets delayed earlier get priority later (“catch up”).
  - Packets that were serviced early are slowed down.
  - Balances delays → results in **consistent end-to-end delay**.

Admission Control in QoS(1/2)

- **Definition:**
  - Process of deciding whether to accept or reject a new flow with QoS requirements.
  - Ensures sufficient resources are reserved to guarantee QoS.
- **Mechanism:**
  - User/application offers a flow with QoS needs → network evaluates capacity.
  - If accepted, **routers reserve resources** (bandwidth, buffers, CPU cycles) along the path.
  - A single congested router can **break the guarantee**.
- **Routing Considerations:**
  - Traditional routing uses the “best path,” which may cause rejections.
  - **QoS Routing**: selects alternative paths with spare capacity.
  - Possible to **split traffic** over multiple paths (equal-cost or capacity-based).

Admission Control in QoS(2/2)

- **Complexities in Decision Making:**
  - Applications often know only **bandwidth needs**, not buffers/CPU cycles.
  - Flow descriptions must be **translated into router resource requirements**.
  - Some apps tolerate **occasional missed deadlines**, others require strict guarantees.
  - **Hard guarantees**: expensive (require worst-case capacity).
  - **Statistical guarantees**: cheaper, sufficient for many applications.
- **Flexibility in Applications:**
  - Some flows may **negotiate QoS parameters**.
  - Example: video app might lower frame rate (30 fps → 25 fps), reduce resolution, or adjust audio.
  - Others may be rigid and not adjustable.

Flow Specification in QoS

- **Purpose:**
  - Describes the flow with **specific, negotiable parameters**.
  - Needed because many parties (sender, receiver, routers) participate in negotiation.
- **Process:**
  - **Sender (e.g., video server)** proposes initial flow specification.
  - As it propagates along the route:
    - Each router examines parameters.
    - Routers may **reduce** capabilities (e.g., lower data rate), but not increase.
  - At the receiver's end, the final parameters are established.
- **Example Parameters (based on RFCs 2210 & 2211 – Integrated Services):**
  - **Token bucket rate** → Maximum sustained transmission rate (long-term average).
  - **Token bucket size** → Maximum burst size allowed in short intervals.

Parameter	Unit
Token bucket rate	Bytes/sec
Token bucket size	Bytes
Peak data rate	Bytes/sec
Minimum packet size	Bytes
Maximum packet size	Bytes

Figure 5-32. An example flow specification.

Flow Specification – Additional Parameters

- **Peak Data Rate**
  - Maximum rate tolerated, even for very short bursts.
  - Sender must never exceed this rate.
- **Packet Size Limits**
  - **Minimum size**:
    - Ensures routers handle realistic packets per second.
    - Example: 10,000 packets/sec of 1 KB is manageable, but 100,000 packets/sec of 50 bytes may overload CPU despite lower bandwidth.
  - **Maximum size**:
    - Restricted by network technology.
    - Example: Ethernet → max packet size = 1500 bytes.
- **Router Resource Translation**
  - Flow specification must be mapped to bandwidth, buffers, CPU cycles.
  - Not just link speed ÷ packet size → statistical fluctuations cause idle periods, so actual reservations must account for variability.

Integrated Services

- **RSVP—The Resource reSerVation Protocol**

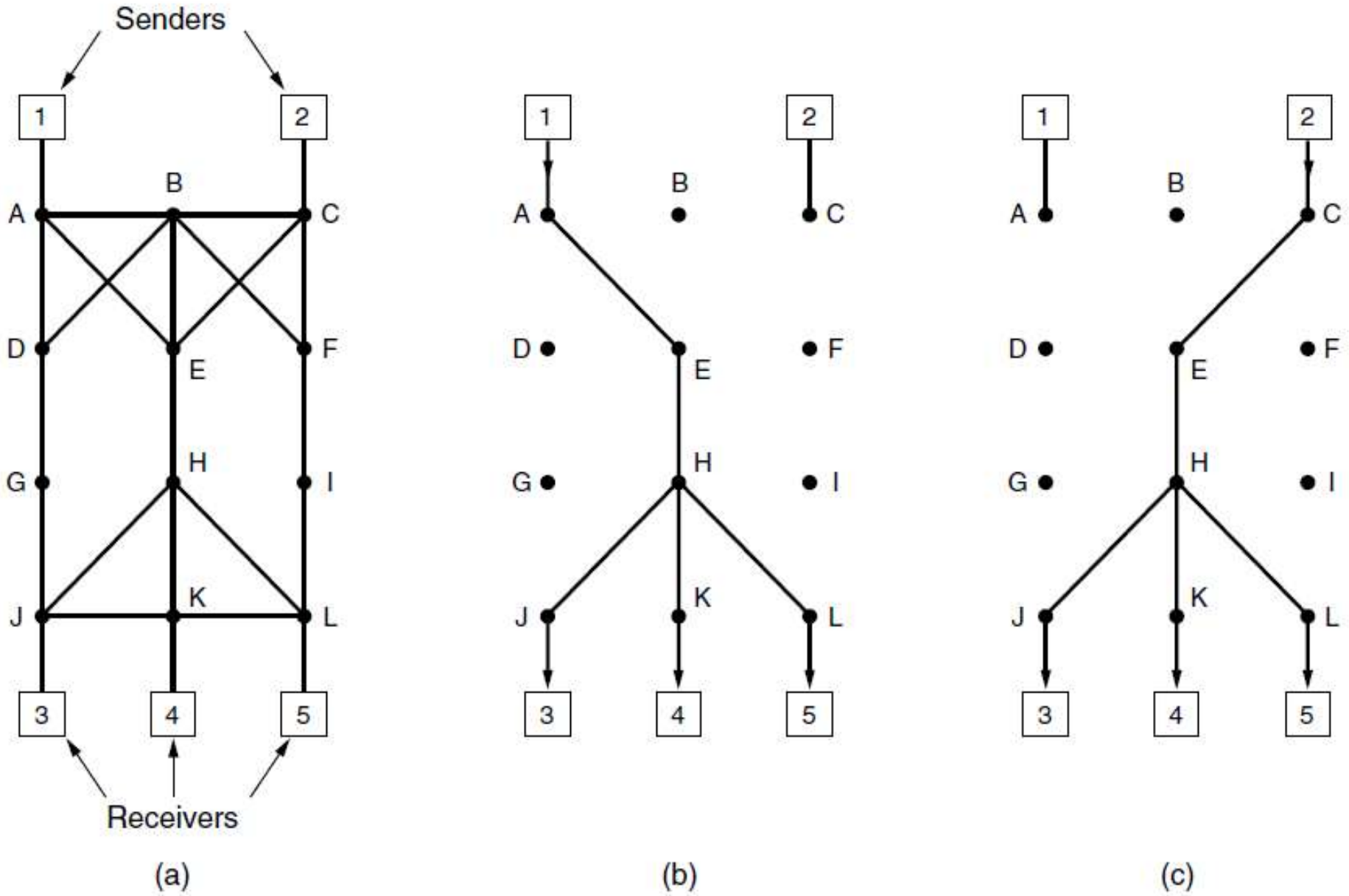


Figure 5-34. (a) A network. (b) The multicast spanning tree for host 1. (c) The multicast spanning tree for host 2.

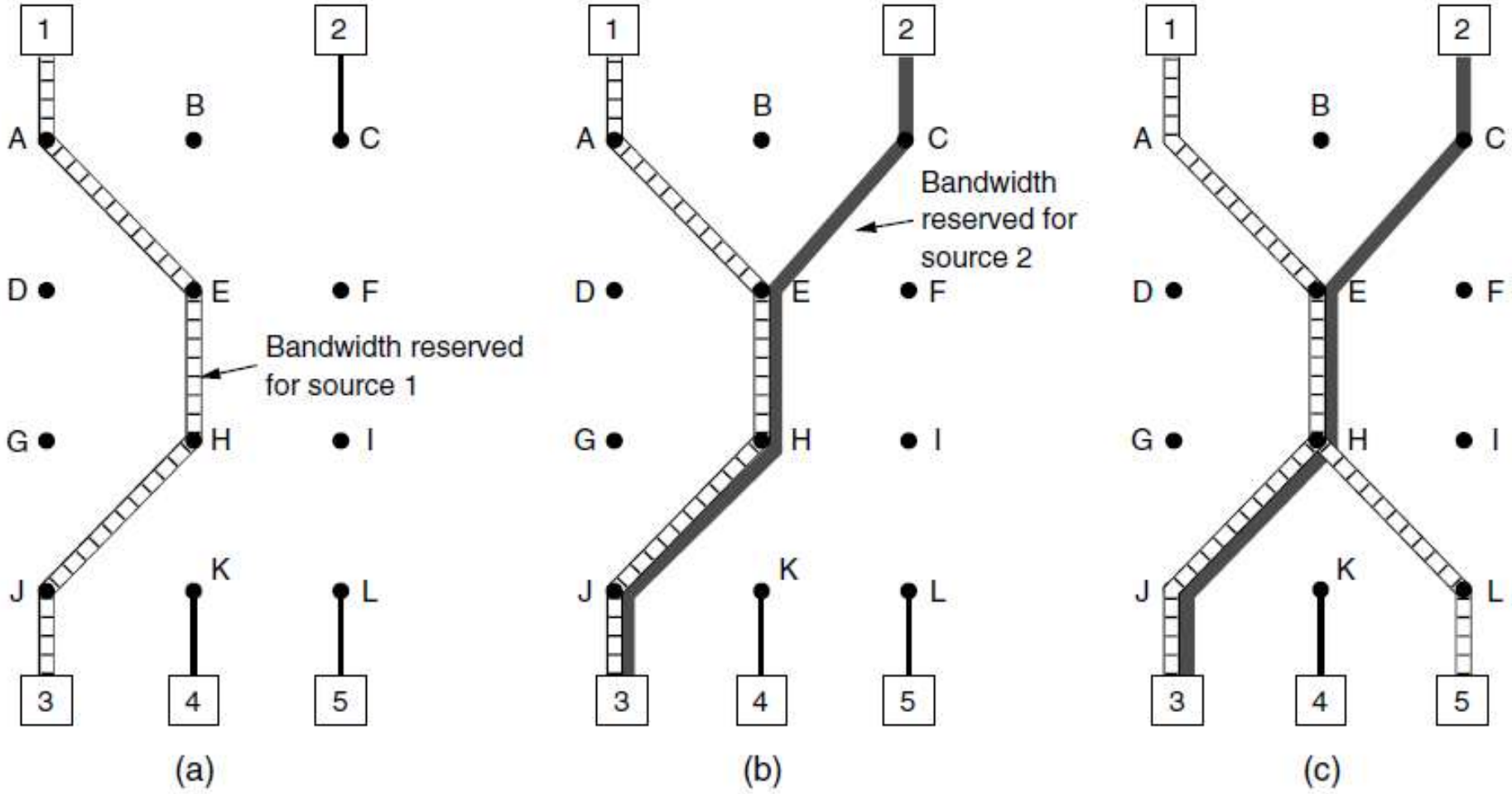


Figure 5-35. (a) Host 3 requests a channel to host 1. (b) Host 3 then requests a second channel, to host 2. (c) Host 5 requests a channel to host 1.

Differentiated Services

- **Flow-Based QoS (Integrated Services)**
- **Pros**:
  - Provides good QoS by reserving exact resources along the route.
- **Cons**:
  - Requires advance setup for each flow → not scalable for thousands/millions of flows.
  - Routers must maintain **per-flow state** → vulnerable to crashes.
  - Complex router-to-router exchanges and major router code changes needed.
  - Very limited real-world deployment.

Class-Based QoS (Differentiated Services – DiffServ)

- **IETF's simpler alternative** (RFC 2474, 2475, etc.).
- **Key features**:
  - Works within an **administrative domain** (e.g., ISP, telco).
  - **No advance setup / per-flow state** required.
  - **Packets are marked** at entry with their service class (in IPv4/IPv6 DS field).
  - Classes correspond to **Per-Hop Behaviors (PHB)** → defines treatment at each router.
- **Examples**:
  - Premium service vs regular service.
  - Traffic shaping per class (e.g., leaky bucket, drain rate).
- **Business model**:
  - ISPs may charge extra for premium packets or offer fixed quotas of premium packets.
- **Advantage**:
  - Scalable, easy to implement, avoids end-to-end negotiations.

Real-World Analogy for Class-Based Service

- **Package delivery** → Overnight vs two-day vs three-day shipping.
- **Airlines** → First class, business class, economy.
- **Trains** → AC class vs sleeper class.
- **Subways** → Premium vs regular service.
- 👉 The idea: **different service categories with shared resources inside each class**.

- **Flow-based QoS** = **fine-grained, strict guarantees** (but complex and unscalable).

- **Class-based QoS** = **coarse-grained, easier to scale** (but no per-flow guarantees).

Aspect	Flow-Based QoS (Integrated Services)	Class-Based QoS (Differentiated Services)
Resource Allocation	Each flow (e.g., each phone call) gets **dedicated resources** with guarantees.	All phone calls share the **same telephony class resources**.
Guarantees	Per-flow guarantees → predictable delay, jitter, bandwidth.	Only class-level guarantees → no guarantee for an individual call.
Isolation	Resources for one flow cannot be stolen by others.	Resources are protected at the **class level** (e.g., telephony vs web), but calls within the same class compete.
Scalability	Poor – needs per-flow state and setup at every router.	Scales well – only a few classes, no per-flow state.
Complexity	High (router-to-router negotiation, signaling protocols).	Low (just marking packets with service class).

Expedited Forwarding (EF) – RFC 3246

- **Concept**:
  - Two classes of service: **regular** and **expedited**.
  - Most traffic = **regular**, small fraction = **expedited**.
  - Expedited packets move through the network **as if no other traffic exists**.
- **Goals of EF**:
  - **Low loss**
  - **Low delay**
  - **Low jitter**
  - Ideal for **real-time applications** like **VoIP**.
- **Visualization (Two-Tube Model)**:
  - One physical link, but **two logical pipes**:
    - Pipe 1 → Regular traffic
    - Pipe 2 → Expedited traffic (reserved bandwidth)
  - Ensures expedited traffic is isolated from congestion in regular traffic.

Implementation

1. **Classification**: Packets are classified as **expedited** or **regular**.
2. **Marking**: Marked accordingly (using the DS field in IP header).
3. **Where it happens**:
    - At the **sending host** (networking software/OS marks VoIP packets).
    - Or at the **ingress router** (first router in the network).

- **Advantage of marking at host**:
    - Host has **better knowledge of flows**.
    - Existing applications need **no modification** (OS handles marking).
    - Example: **VoIP packets** are commonly marked for EF by hosts today.
- **If EF not supported by network**:
    - Packets are simply treated as **regular traffic**.
    - No harm done (graceful fallback).

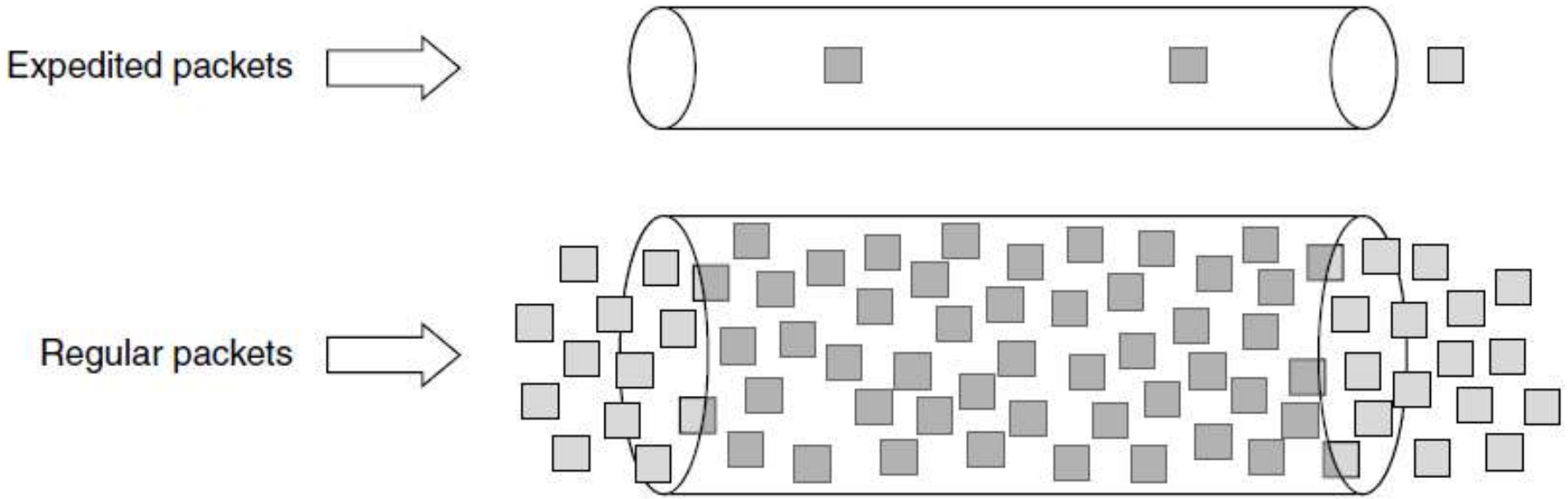


Figure 5-36. Expedited packets experience a traffic-free network.

- **Assured Forwarding (AF) – RFC 2597**
- **Concept**:
  - Provides **four priority classes** (e.g., **Gold, Silver, Bronze, + one more**).
  - Each priority class has its **own resources**.
  - Within each priority class, packets are further divided into **three discard classes**:
    - **Low discard** (most protected)
    - **Medium discard**
    - **High discard** (most likely to be dropped during congestion)
  - → **Total = 4 × 3 = 12 service classes**.

- **Packet Processing Steps**:
  - **Classification**:
    - Packets classified into one of the **four priority classes**.
    - Can be done at the **sending host** or at the **ingress router**.
    - Operator may **limit the rate** of high-priority packets as part of service policy.
  - **Discard Class Assignment (Policing)**:
    - A **traffic policer** (e.g., **token bucket**) is applied.
    - Packets are tagged based on how they fit traffic limits:
      - Within small bursts → **Low discard**.
      - Slightly over small bursts → **Medium discard**.
      - Far beyond large bursts → **High discard**.
  - **Encoding**:
    - Each packet is marked with its **priority + discard class**.
  - **Scheduling in Routers**:
    - Routers use a **packet scheduler** that distinguishes between classes.
    - Common method: **Weighted Round Robin (WRR)**.
      - Ensures higher-priority and low-discard packets are forwarded first.

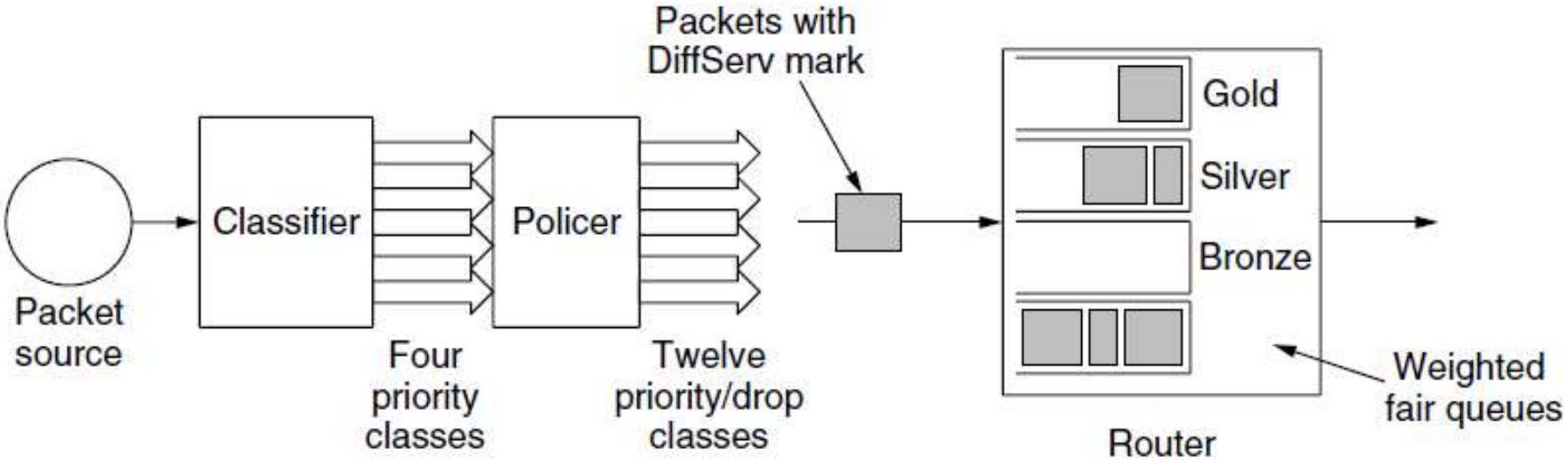


Figure 5-37. A possible implementation of assured forwarding.

- **WFQ in Assured Forwarding**
- **Bandwidth Sharing**
  - Routers apply **Weighted Fair Queueing (WFQ)** across the **four priority classes**.
  - **Higher priority classes → higher weights** → more bandwidth share.
  - Example: if weights **double** from one class to the next:
    - Class 1 (lowest) → 1 unit
    - Class 2 → 2 units
    - Class 3 → 4 units
    - Class 4 (highest) → 8 units
    - → Class 4 gets **8× more bandwidth** than Class 1, but **no class is starved**.
- **Handling Congestion (Within a Class)**
  - Packets of the **same priority class** are further managed by their **discard class**.
  - Uses **RED (Random Early Detection)**:
    - **High discard packets** dropped first as congestion begins.
    - **Low discard packets** kept longer, increasing their delivery probability.
    - Prevents buffer overflow by **early, selective dropping**.
- **Outcome**
  - **Fairness**: Lower classes still get some bandwidth.
  - **Efficiency**: High-priority flows see better throughput and lower delay.
  - **Flexibility**: Operators can tune **weights + RED thresholds** to match SLAs (Gold, Silver, Bronze, etc.).